

第 1 课 · 面试题库（先认清你的机器）

母题索引

| 母题                  | 一句话                                               | 覆盖题号                           |
|---------------------|---------------------------------------------------|--------------------------------|
| M1 选对指标             | 同一个命令，看错那一列等于没看（available 不是 free；df -i 不是 df -h） | 5, 6, 13, 14, 22               |
| M2 先分类，再动手          | 先分清"服务问题 / 权限问题 / 资源问题"，再决定查什么                    | 1, 20, 21, 22                  |
| M3 让机器自己说话          | 结论必须来自命令输出，不来自记忆或感觉                               | 1, 2, 3, 9, 10, 12, 17, 18, 19 |
| M4 共享边界             | 内核可共享、用户态不可共享 —— 一条原理解释容器/虚拟机的全部差异                | 4, 16, 23, 24                  |
| M5 看不见的占用           | 删了没释放（句柄）、被盖住（挂载覆盖）、数量满（inode）                    | 6, 7, 15, 21                   |
| M6 版本决定兼容           | 版本不是背景信息，是故障的第一嫌疑对象                               | 3, 8, 16                       |
| M7 暴露面 = 监听地址 + 防火墙 | "能不能被访问"永远是两件事叠加                                  | 11, 20                         |

问题清单

A 组 · 基础必答（答不上来会被判定为"只会敲命令"）

- ❑ 1. 【★★★】你接手一台陌生的 Linux 服务器，第一步做什么？说出完整的体检清单。
- ❑ 5. 【★★★】free -h 里 free、buff/cache、available 三列分别是什么？为什么 free 很小系统却不卡？
- ❑ 6. 【★★★】df -h 和 df -i 的区别？什么情况下"磁盘还有空间却写不进文件"？
- ❑ 7. 【★★★】du 和 df 的结果对不上，可能有哪些原因？
- ❑ 2. 【★★】怎么判断这台机器是物理机、虚拟机还是容器？
- ❑ 3. 【★★】怎么查看系统版本？为什么更推荐 /etc/os-release 而不是 lsb\_release -a？
- ❑ 4. 【★★】uname -r 和 cat /etc/os-release 分别告诉你什么？两者能互相替代吗？
- ❑ 8. 【★★】怎么查看 CPU 核数？nproc、lscpu、/proc/cpuinfo 有什么区别？
- ❑ 9. 【★★】lsblk 看什么？怎么知道分区表是 MBR 还是 GPT？
- ❑ 10. 【★★】怎么查看机器的 IP？为什么现在的教程都用 ip 而不用 ifconfig？
- ❑ 11. 【★★】127.0.0.1 和 0.0.0.0 有什么区别？
- ❑ 12. 【★★】怎么看这台机器是不是跑在云上？是哪家云？

B 组 · 进阶

- ❑ 13. 【★★★】buff/cache 能手动释放吗？生产上要不要释放？
- ❑ 14. 【★★★】swap 是什么？vm.swappiness 调到 10 有什么影响？为什么 K8s 节点要求关 swap？
- ❑ 15. 【★★★】inode 是什么？ext4 的 inode 数量能在线扩容吗？
- ❑ 16. 【★★】内核版本对排障有什么影响？举一个真实例子。
- ❑ 17. 【★★】怎么查看机器开机多久了、什么时候启动的、启动花了多长时间？
- ❑ 18. 【🔧】怎么判断一块盘是 SSD 还是 HDD？
- ❑ 19. 【☆】一台机器的"资产信息"你会记录哪些字段？为什么要记？

C 组 · 排障场景题

- ❑ 20. 【★★★】你在 Ubuntu 上写了一条 iptables 规则，iptables -L 看得到，但就是不生效，怎么排查？
- ❑ 21. 【★★★】df -h 显示根分区 100% 用了，但 du -sh / 加起来只有 40G，怎么定位？
- ❑ 22. 【★★★】一台机器 16G 内存，free 只剩 200M，客户说"系统要挂了"，你怎么回复？
- ❑ 23. 【★★】你在容器里 cat /etc/os-release 显示的是 Debian，宿主机是 Ubuntu，为什么？

D 组 · 追问链（面试官顺藤摸瓜，一题变四题）

- ❑ 24. 【★★】从"怎么判断是虚拟机还是物理机"出发，可能被一路追问到：systemd-detect-virt 的原理是什么？→ 不用它还能怎么看？→ 阿里云 ECS 为什么显示 kvm？→ 容器为什么也能被识别出来？

参考答案

【★★★】A1 · 接手一台陌生服务器，第一步做什么？

- 结论：先取证，不动任何配置。搞清楚"我是谁、有什么资源、我在哪、有没有已经坏的东西"。
- 清单（照着念就是完整答案）：cat /etc/os-release（系统版本）→ uname -r（内核）→ nproc && free -h && df -h /（CPU/内存/磁盘）→ systemd-detect-virt（物理/虚拟/容器）→ ip -c a（网络）→ systemctl --failed（有没有起不来的服务）→ uptime（负载和运行时长）
- 加分点：① 生产上先看变更记录和监控入口，再敲命令 —— 十次故障八次是最近改过东西；② 改任何文件前先 cp -a 备份；③ 把这套体检写成脚本（第 3 月就会做，正好呼应）。

【★★】A2 · 怎么判断物理机、虚拟机还是容器？

- 命令：systemd-detect-virt
- none = 物理机（双系统也属于物理机）
- kvm / vmware / oracle / microsoft = 虚拟机（分别是 KVM/云、VMware、VirtualBox、Hyper-V）

- `docker / lxc / podman` = 容器
- **展开**：它检测三样东西 —— DMI/SMBIOS 里的厂商串、CPU 的 "hypervisor present" 标志位、容器的特征文件（如 `/.dockerenv`、`cgroup` 信息）。
- **加分点**：`systemd-detect-virt --vm / --container` 可以只测某一类，**脚本里用它做条件判断**；`/proc/1/` 也是容器排障的关键线索。

### 【★★】A3 • 怎么查看系统版本？为什么用 `/etc/os-release`？

- **结论**：用 `cat /etc/os-release`。它是 freedesktop 标准，**所有主流发行版都有**，而且是纯文本、好解析。
- **为什么不用 `lsb_release`**：`lsb_release` 属于 `lsb-release` 包，**精简安装和容器镜像里经常没装**，一条"命令不存在"就把你挡在门外。
- **加分点**：脚本里取版本号用 `cat /etc/os-release && echo $VERSION_CODENAME` —— 这也是装 Docker 官方源时的写法。

### 【★★】A4 • `uname -r` 和 `/etc/os-release` 能互相替代吗？

- **不能，是不同层次的信息**：`/etc/os-release` 是**发行版**（用户态：Ubuntu 26.04、代号、包体系）；`uname -r` 是**内核**（内核态：真正跟硬件打交道的那层）。
- **例子**：同一台 Ubuntu 可以换内核；同一个内核可以配不同发行版。
- **加分点**：所以容器里 `uname -r` 与宿主**相同**、`/etc/os-release` 与宿主**不同** —— 这是判断"容器 vs 虚拟机"的实战技巧（见 C23）。

### 【★★★】A5 • `free / buff-cache / available` 三列是什么？

- **一句话**：`available` 才是"现在真正能用的内存"，`free` 只是"一点缓存都没占的裸内存"。
- **为什么 `free` 很小却不卡**：Linux 的设计是**空闲内存等于浪费** —— 内核把暂时没人用的内存拿去做磁盘缓存（`buff/cache`），一有程序要内存，缓存随时让路。
- **怎么做**：看 `available`；理解 `used` 也要连 `buff/cache` 一起看。
- **加分点**：判断内存是否真紧张，还要看 **swap 使用量**和 **OOM 记录**（`dmesg -T | grep -i oom`、`journalctl -k | grep -i oom`）。只看 `free` 一列就下结论，是典型的"看错指标"。

### 【★★★】A6 • `df -h` 和 `df -i` 的区别？

- **两个维度**：`df -h` 看**块空间**（还有多少 G），`df -i` 看**inode 数量**（还能建多少个文件）。
- **inode 是什么**：每个文件占一个 inode，存权限、时间、数据块位置等元数据。**文件很多但都很小时**，空间还剩几百 G，inode 先满 —— 报错通常是 `No space left on device`，但 `df -h` 看着还有空。
- **典型场景**：`session` 文件、邮件队列、海量小日志、缓存小文件、`/tmp` 里几十万个小文件。
- **怎么做**：先 `df -i` 确认，再找哪个目录文件最多：`for d in /var/*; do echo -n "$d "; find $d | wc -l; done`。
- **加分点**：`ext4` 的 inode 数在 `mkfs` 时就固定了，**不能在线扩容**；XFS 动态分配 inode，大目录场景更合适。

### 【★★★】A7 • `du` 和 `df` 对不上，有哪些原因？

- **最常见**：**文件被删除，但进程还拿着句柄**，空间不释放。查：`lssof +l1` 或 `lssof | grep deleted`；解决：重启那个进程，或清空句柄（：`> /proc/PID/fd/N`）。
- **第二种**：**挂载点覆盖** —— 你往挂载目录里写过文件，之后又挂了一块盘上去，原文件被"盖住"，`du` 看不见但空间确实被占。
- **第三种**：**`ext4` 给 `root` 保留的 5% 块**（`tune2fs -l` 看 `Reserved block count`），普通用户写不进，`df` 的数字也会有差异。
- **加分点**：排"磁盘满"必须三条一起跑：`df -h`（分区级）+ `du -sh *`（目录级）+ `lssof +l1`（句柄级）。

### 【★★】A8 • `nproc / lscpu / /proc/cpuinfo` 的区别？

- `nproc`：**当前进程能用的核数**，会受 CPU 亲和性（`taskset`）和 `cgroup` 限制影响 —— 容器里可能比宿主少。
- `lscpu`：**架构级信息** —— 型号、主频、缓存层级、是否支持虚拟化（VT-x/AMD-V）、NUMA。
- `/proc/cpuinfo`：**逐核原始明细**，脚本解析用。
- **加分点**：`nproc` ≠ 物理核数（开了超线程就是逻辑核）；排查"CPU 打满"要用 `mpstat -P ALL 1` 分辨是**单核跑满还是整体跑满**。

### 【★★】A9 • `lsblk` 看什么？MBR 还是 GPT？

- `lsblk -f`：设备名、分区、文件系统类型、UUID、挂载点 —— 看的是**盘的物理布局**。
- **分区表类型它看不到**，要看 `sudo fdisk -l`（找 `Disklabel type: gpt` 或 `dos`）或 `sudo parted -l`。
- **限制差异**：MBR 最多 4 个主分区、单盘 ≤ 2TB；GPT 支持 128 个分区、超大容量，分区表还有备份。
- **加分点**：`lsblk` **看不到"未分配空间"** —— 想扩容前先 `parted -l` 确认后面还有没有空地，这是 1.5④ 扩容决策的关键。

### 【★★】A10 • 查 IP 用什么？为什么淘汰 `ifconfig`？

- 用 `ip -c addr show`（简写 `ip -c a`）。
- **淘汰原因**：`net-tools` 早已停止维护，`ifconfig` 不支持网络命名空间、多 IP 别名、策略路由等现代特性；`iproute2` 一套工具覆盖 `ip`（地址/路由）+ `ss`（连接）+ `tc`（流量控制），性能也更好。
- **加分点**：生产最小化镜像里常常**只有 `iproute2`、没有 `net-tools`** —— 会 `ip` 的人进机房才不会卡住。

### 【★★】A11 • 127.0.0.1 和 0.0.0.0

- **127.0.0.1 (lo)**：回环地址，**只能本机访问**，流量不出网卡。
- **0.0.0.0 (监听所有)**：服务绑在它上面表示接受**所有网卡**的连接，包括以后新增的网卡。
- **安全含义**：服务绑 0.0.0.0 且防火墙放行，就等于**对公网暴露**。所以"能不能被访问"是两件事叠加：**监听地址 + 防火墙/安全组策略**。
- **加分点**：排查"服务起得来但外部连不上"，第一件事就是确认它到底监听 127.0.0.1 还是 0.0.0.0：`ss -lntp`。

## 【★★】A12 • 怎么判断在云上？哪家云？

- `systemd-detect-virt` → `kvm`（云上几乎都是 KVM）只是提示，不足以定是哪家。
- 准的判断：① 看 BIOS/DMI: `sudo dmidecode -s system-product-name`（如 "Alibaba Cloud ECS"、"VMware Virtual Platform"）；② 看元数据服务：阿里云实例内可直接 `curl http://100.100.100.200/latest/meta-data/` 拿实例 ID、地域、私网 IP；AWS 是 169.254.169.254。
- 加分点：元数据服务同时是安全风险点（SSRF 攻击常打它）——提一句“云主机上要限制谁能访问元数据服务”非常加分。

## 【★★★】B13 • buff/cache 能释放吗？

- 能，但不该：`echo 3 > /proc/sys/vm/drop_caches` 可以释放（1 = pagecache、2 = dentries/inodes、3 = 全部）。
- 为什么不该：缓存是“用空闲内存换 IO 性能”，释放后系统要立刻从磁盘重读，IO 会瞬时抖动，业务可能变慢。只有做基准测试、需要干净起点时才用。
- 正确姿势：内存真紧张要看 `available`、`swap` 使用和 OOM 记录，而不是“把缓存清掉看着舒服”。

## 【★★★】B14 • swap 和 swappiness

- swap 是什么：把不常用的内存页写到磁盘上，腾出物理内存。速度比内存慢几个数量级，它是“保命”不是“提速”。
- swappiness: 0~100，值越大越倾向用 swap。调到 10 = 尽量不用，只在快要 OOM 时才动它。
- 为什么 K8s 要求关 swap: kubelet 默认 `--fail-swap-on=true`，因为 swap 会让内存用量、QoS 等级、驱逐 (eviction) 判断失真，延迟不可控，调度器无法准确评估节点剩余资源。
- 关掉的正确做法：`sudo swapoff -a` + 注释 `/etc/fstab` 里的 swap 行（否则重启又回来）。
- 加分点：本计划用的 k3d 不需要关 swap —— 节点跑在容器里，内存由宿主 Docker 管，不依赖宿主 swap 状态。“知道边界在哪”比背结论更值钱。

## 【★★★】B15 • inode 是什么？能在线扩容吗？

- inode: 文件系统给每个文件分配的元数据结构（权限、属主、时间戳、大小、数据块指针）；文件名只是目录里指向 inode 的一条记录。
- ext4: inode 总数在 `mkfs.ext4` 时按比例固定（约每 16KB 一个），不能在线扩容 —— 满了只能清理、重建或迁移到新盘。
- XFS: 动态分配 inode，不容易撞上这个限制，海量小文件场景更合适。
- 加分点：`df -i` 和 `df -h` 要成对看，这是“磁盘满”两类故障的分水岭。

## 【★★】B16 • 内核版本对排障的影响

- 结论：内核版本决定了你能用哪些特性、会遇到哪些兼容坑，排障前必须先记下来。
- 真实例子：新内核 + 厂商 Agent/驱动没跟上 → 监控采集不到、网卡驱动异常；`iptables` 与 `iptables-nft` 前端混用 → 规则“看着对但不生效”；依赖新内核特性的容器/编排工具有行为差异。
- 重要澄清：`cgroup v2` 和 `nftables` 早就是 Ubuntu 的默认，不是新版本才有的 —— 把它们当“新版特性”去找原因是方向性错误。
- 加分点：搜报错时带上系统版本号和内核号（`uname -r`），命中率完全不同。

## 【★★】B17 • 开机多久了 / 启动花了多长时间

- `uptime`：第一项就是运行时长，后面是 1/5/15 分钟负载。
- `who -b` 或 `uptime -s`：系统启动时刻。
- `systemd-analyze`：内核 + 用户态总耗时；`systemd-analyze blame`：哪个服务最慢；`systemd-analyze critical-chain`：关键路径。
- 加分点：“重启后服务没起来”的经典排查起点就是 `systemctl --failed + systemd-analyze blame`。

## 【🔧】B18 • 怎么判断 SSD 还是 HDD

- `lsblk -d -o NAME,ROTA,SIZE`：ROTA=0 固态、1 机械。
- `cat /sys/block/sda/queue/rotational`：同上，脚本里好用。
- 更进一步：`sudo smartctl -a /dev/sda` 看型号、通电时间、健康告警（需装 `smartmontools`）。
- 加分点：数据库/容器所在盘的介质类型直接影响 IO 性能判断 —— 看到 `iowait` 高，先确认是不是机械盘。

## 【☆】B19 • 资产信息记什么？为什么要记

- 字段：主机名、内/公网 IP、系统与内核版本、CPU/内存/磁盘、角色（web/db/缓存）、机房或云账号地域、负责人、到期时间、上面的关键服务。
- 为什么：这就是轻量版 CMDB。故障时能立刻算出影响面（谁依赖它、它依赖谁）；变更前能算出风险（动它影响几个业务）；成本上能回答“这台机器在干嘛、能不能下线”。
- 加分点：面试时说“我做了巡检脚本，输出结构化清单（主机名/IP/负载/磁盘/关键服务状态）”，等于证明你有资产意识。

## 【★★★】C20 • iptables 规则“看得见但不生效”

- 按顺序查：① 前端混用：`update-alternatives --display iptables` 看当前是 `legacy` 还是 `nft` —— 两种前端各有一套规则表，你写的和生效的可能不是同一套；② 规则顺序：`iptables` 从上到下匹配，命中即停，被前面的 `ACCEPT/DROP` 拦住了；③ 表/链对不对：filter 还是 nat、INPUT 还是 FORWARD；④ 是否被 Docker 绕过：Docker 会往 nat/FORWARD 插自己的链，`ufw` 的规则拦不住容器映射的端口（要写在 `DOCKER-USER` 链里）；⑤ 持久化：重启就没了，需要 `netfilter-persistent`。
- 看真相：`sudo nft list ruleset` 看内核里实际生效的规则集。
- 加分点：先给结论“大概率是 `iptables-nft` 与 `legacy` 混用，或 Docker 绕过了 `ufw`”，再说怎么验证 —— 面试官要的是判断力，不是命令罗列。

## 【★★★】C21 • df 满但 du 加起来不多

- 第一步：`lsdf +L1`（或 `lsdf | grep -i deleted`）找已删除但被进程占着句柄的文件 —— 这是最常见原因。

- **第二步**：确认有没有**挂载点覆盖**（挂载后在原目录写过文件），可用 `mkdir /tmp/x && mount --bind / /tmp/x` 再看。
- **第三步**：查 ext4 保留块和稀疏文件（少见）。
- **解决**：重启持有句柄的进程（最干净），或临时清空句柄：`> /proc/PID/fd/N`；然后给日志加轮转，防止复发。
- **加分点**：一句话总结 —— “**df 看的是文件系统的账本，lsof 看的是谁还攥着账本上的钱。**”

#### 【★★★】C22 • free 只剩 200M，怎么回复客户

- **别慌，先取证**：`free -h` 看 **available**（才是真能用的）；`vmstat 1 5` 看 si/so 是否在 swap 反复换页；`dmesg -T | grep -i oom` 看有没有 OOM 记录；`ps aux --sort=-%mem | head` 看谁在吃。
- **回复要有数据**：例如“available 还有 10G，swap 未使用，负载正常，无 OOM 记录 —— 当前是健康的；free 小是因为内核把空闲内存用作了磁盘缓存”。
- **加分点**：给客户/领导结论时**先说“是否影响业务”**再讲技术细节；同时提出“我加一条 available 低于 10% 的告警”，把被动答疑变成主动防护。

#### 【★★】C23 • 容器里 os-release 是 Debian，宿主机是 Ubuntu

- **原因**：容器**共享宿主机内核，但有自己的根文件系统**（来自镜像）。`/etc/os-release` 是用户态文件，跟着镜像走；`uname -r` 是内核信息，跟着宿主走。
- **一句话验证**：容器里 `uname -r` 与宿主**相同**、`/etc/os-release` 与宿主**不同** → 说明是容器，不是虚拟机。
- **加分点**：这就解释了“为什么容器里装不了内核模块、改不了内核参数” —— 内核是共用的。

#### 【★★】D24 • 追问链：虚拟机 / 物理机 / 容器怎么识别

- **systemd-detect-virt 的原理**：三路取证 —— ① DMI/SMBIOS 里的厂商串（VMware/QEMU/Microsoft 会明说）；② CPU 的 hypervisor present 位（CPUID）；③ 容器特征（`/.dockerenv`、`/run/.containerenv`、`cgroup` 路径）。
  - **不用它怎么看**：`sudo dmidecode -s system-product-name、lscpu | grep -i hypervisor、cat /proc/1/cgroup`（容器里能看到 docker/kubepods 字样）、`ls -a / | grep docker`。
  - **为什么阿里云 ECS 显示 kvm**：阿里云底层用 KVM 做虚拟化，你买到的是 KVM 虚拟机 —— 显示 `kvm` 是正确的，不代表厂商就叫 KVM。
  - **容器为什么也识别得出来**：它检测的是“进程看到的运行环境”，容器有独立的 PID/mount 命名空间和标志文件，所以能被判定为 `docker/lxc`。
  - **收尾话术**：补一句 —— “这一类判断我习惯写进体检脚本，接手一台机器就跑一遍出报告。”把知识点变成工程习惯，是面试里最容易被记住的部分。
-